



ArtMintfy

Omni-Chain NFT Deployment Protocol

Technical Whitepaper | Version 1.0 | 2024

Table of Contents

1	Abstract
2	Introduction & Problem Statement
3	Protocol Architecture
4	Smart Contract Design
5	IPFS & Decentralised Storage
6	Multi-Chain Deployment Strategy
7	Token Standard & Metadata
8	Security Model
9	Developer Integration
10	Roadmap
11	Conclusion

1

ABSTRACT

ArtMintfy is an open, permissionless protocol that enables artists and developers to deploy ERC-721 NFT collections deterministically across multiple EVM-compatible blockchains in a single transaction flow. By combining a factory-pattern smart contract, IPFS-pinned media, and a browser-based omni-chain orchestration layer, ArtMintfy removes the technical and economic barriers that have historically prevented independent creators from participating in the multi-chain NFT ecosystem.

Chains Supported 6+ Celo, Base, Arbitrum, Polygon, Optimism, Lisk	Deployment Model CREATE2 Deterministic address across all chains	Storage IPFS Pinata-pinned, content-addressed media	Contract Standard ERC-721 Enumerable + Ownable + Custom metadata
--	---	--	---

2 INTRODUCTION & PROBLEM STATEMENT

2.1 The Multi-Chain Reality

The NFT market has fragmented across dozens of EVM-compatible networks. Each chain offers different trade-offs in cost, speed, and community reach. Creators who wish to maximise their distribution must either deploy separate contracts manually on each chain — a process requiring technical expertise and significant time — or accept the limited reach of a single-chain strategy.

2.2 Pain Points Addressed

- **Technical complexity:** Deploying and verifying contracts on multiple networks is error-prone and time-consuming.
- **Address inconsistency:** Contracts deployed naively receive different addresses on each chain, complicating cross-chain indexing.
- **Media permanence:** Centralised image hosting creates long-term availability risk for NFT metadata.
- **High entry costs:** Gas fees on premium chains (Ethereum mainnet, Base) price out independent artists.
- **UX friction:** Existing tools require separate workflows per chain with no unified deployment dashboard.

2.3 ArtMintfy's Answer

ArtMintfy solves these problems with a three-layer architecture: a unified browser interface that guides the user through artwork upload, collection configuration, and multi-chain deployment; a deterministic factory contract deployed at the same address on every supported chain; and IPFS-pinned media that ensures artwork availability independent of any centralised service.

3 PROTOCOL ARCHITECTURE

The ArtMintfy protocol is composed of three tightly integrated layers. Each layer is independently replaceable, allowing the protocol to evolve without breaking existing deployments.

Layer	Component	Technology	Role
Presentation	Omni-Deploy UI	Next.js 14 + TypeScript	User-facing wizard for upload, config & deploy
Storage	IPFS Pinning	Pinata Cloud API	Permanent content-addressed media storage
Settlement	Factory Contract	Solidity 0.8.x + CREATE2	Deterministic ERC-721 deployment on every chain

3.1 Presentation Layer

The front-end is a Next.js 14 application using the App Router. Client components handle wallet connection via MetaMask's EIP-1193 provider, network switching via `wallet_addEthereumChain`, and contract interaction via ethers.js v5. The four-step wizard (wallet → artwork → metadata → chain selection) enforces sequential validation, preventing deployment until all prerequisites are satisfied.

3.2 Storage Layer

Artwork is uploaded to IPFS via the Pinata pinning service before any on-chain transaction is initiated. The resulting CID (Content Identifier) is used as the token URI for all deployed collections. Because IPFS addresses content by hash rather than location, the artwork is guaranteed to be identical across every chain where the collection is deployed, and remains accessible as long as at least one node pins the content.

3.3 Settlement Layer

The factory contract is the protocol's core primitive. It is deployed at the same address on every supported chain and exposes a single entry point: `deployDeterministic`. This function uses CREATE2 to derive a deterministic NFT contract address from the collection parameters and the deployer's address, ensuring the collection contract occupies the same address on every chain.

4

SMART CONTRACT DESIGN

4.1 Factory Contract

The factory exposes two public functions. *deployDeterministic* takes the collection name, symbol, IPFS URI, and a bytes32 salt derived from the deployer's address. It deploys a new ERC-721 contract via CREATE2 and records the address in an internal registry. *getDeployedNFTs* returns the full list of deployed collection addresses, enabling the ArtExplorer UI to enumerate all collections without an off-chain indexer.

Factory Address

0xE66Ac37142f2e15e96E604BF5B861a304C4fedC4 — deployed at this address on all supported chains. The address is identical across networks because the factory itself was deployed using a deterministic deployment script.

4.2 NFT Collection Contract

Each deployed collection inherits from OpenZeppelin's ERC721Enumerable and Ownable. The collection owner — the wallet that called *deployDeterministic* — retains the ability to *safeMint* additional tokens. Each token stores a custom name and symbol alongside its URI, enabling per-token metadata differentiation within a single collection.

4.3 Deterministic Addressing with CREATE2

The salt passed to CREATE2 is the keccak256 hash of the deployer's Ethereum address. This means a given wallet will always receive the same collection contract address on every chain, provided the factory address and bytecode are identical. This property enables seamless cross-chain asset indexing and simplifies user-facing address display.

Property	Value
Salt derivation	keccak256(deployer_address)
Bytecode	Identical across all chains (audited build artifact)
Factory address	0xE66Ac37142f2e15e96E604BF5B861a304C4fedC4
Result	Same collection address on every supported chain

5 IPFS & DECENTRALISED STORAGE

ArtMintfy uses the InterPlanetary File System (IPFS) as its canonical storage layer. IPFS is a content-addressed distributed file system: every file is identified by a cryptographic hash of its contents (the CID), not by a URL. This means the artwork cannot be altered after upload without changing its CID — and therefore its on-chain URI — providing immutability guarantees equivalent to the blockchain itself.

5.1 Upload Flow

- User selects a file (PNG, JPG, GIF, or SVG, up to 50 MB) in the Omni-Deploy interface.
- The file is submitted to the Pinata API with a JWT bearer token stored as an environment variable.
- Pinata pins the file to IPFS and returns an IpfsHash (the CID).
- The CID is stored in component state and passed as the URI argument to `deployDeterministic`.
- All deployed collections on all chains reference the same CID, guaranteeing media consistency.

5.2 Resolution

The ArtExplorer resolves token URIs at display time. URIs prefixed with *ipfs://* are rewritten to *https://ipfs.io/ipfs/* for browser-friendly display. Raw CIDs without a scheme prefix are also rewritten. HTTP URIs are passed through unchanged. This resolution strategy is forward-compatible with custom IPFS gateways and future decentralised gateway networks such as Protocol Labs' Saturn.

6

MULTI-CHAIN DEPLOYMENT STRATEGY

The omni-chain deployment flow is the protocol's most distinctive feature. Rather than requiring the user to switch networks and re-submit transactions manually, ArtMintfy automates the entire sequence within a single browser session.

6.1 Deployment Sequence

1	Salt computation — keccak256(signer.getAddress()) computed client-side.
2	Chain iteration — For each selected chain, wallet_addEthereumChain switches the network.
3	Factory call — deployDeterministic is called with name, symbol, URI, salt.
4	Confirmation — tx.wait() blocks until the transaction is mined; status updates to success.
5	Retry safety — Already-succeeded chains are skipped on retry, preventing duplicate deployments.

6.2 Supported Networks

Network	Chain ID	Native Token	Explorer	Factory Deployed
Celo Mainnet	42220	CELO	celoscan.io	Yes
Base	8453	ETH	basescan.org	Yes
Arbitrum One	42161	ETH	arbiscan.io	Yes
Polygon	137	POL	polygonscan.com	Yes
Optimism	10	ETH	optimistic.etherscan.io	Yes
Lisk	1135	LSK	blockscout.lisk.com	Yes
Botchain	968	BOT	scan.bohr.life	Configurable

7 TOKEN STANDARD & METADATA

All collections deployed through ArtMintfy conform to the ERC-721 Non-Fungible Token Standard (EIP-721) with the optional ERC721Enumerable extension. This ensures compatibility with all major NFT marketplaces, wallets, and indexers.

7.1 On-Chain Metadata Extensions

ArtMintfy extends the base ERC-721 interface with per-token custom name and symbol storage. The *getCustomName* and *getCustomSymbol* view functions allow marketplaces and analytics tools to surface richer metadata without relying solely on the token URI. The *getAllMintedSuccessfully* function returns the full list of minter addresses, enabling on-chain reputation and airdrop mechanics.

7.2 Metadata Schema

Token URIs point to IPFS-hosted JSON conforming to the OpenSea metadata standard. The image field references the original artwork CID. Additional attributes can be added by the collection owner post-deployment via a metadata update transaction, with the *BatchMetadataUpdate* event notifying indexers of the change.

8 SECURITY MODEL

8.1 Smart Contract Security

- **OpenZeppelin base contracts:** All collection contracts inherit from audited OZ implementations of ERC721, ERC721Enumerable, and Ownable.
- **Ownable access control:** safeMint is restricted to the contract owner, preventing unauthorised minting.
- **Deterministic addressing:** CREATE2 addresses are pre-computable, allowing off-chain verification before deployment.
- **No upgradeability:** Collection contracts are immutable. Metadata updates are limited to the tokenURI, not contract logic.

8.2 Front-End Security

- **Environment variable isolation:** The Pinata API key is stored as NEXT_PUBLIC_NFT_STORAGE_KEY and never sent server-side.
- **Input validation:** The deploy button is blocked until all required fields pass client-side validation.
- **Network verification:** The provider's chainId is verified against the expected value before each contract call.
- **No private key storage:** All signing is delegated to MetaMask. ArtMintfy never has access to user private keys.

8.3 Known Limitations

- The Pinata JWT is exposed in the browser bundle via NEXT_PUBLIC_. Production deployments should proxy upload requests through a server-side API route.
- The salt is derived solely from the deployer address. Users who lose access to their wallet cannot re-derive the same collection address.
- Factory deployment is currently permissionless. A governance mechanism for factory upgrades is planned for v2.

9 DEVELOPER INTEGRATION

ArtMintfy is fully open-source. Developers can fork the repository, add custom chains, and extend the factory contract without permission. The integration surface is minimal by design.

9.1 Adding a New Chain

- Deploy the factory contract at 0xE66Ac37142f2e15e96E604BF5B861a304C4fedC4 on the target chain (use a vanity deployer to match the address).
- Add the chain config to lib/networks.js with a hex chainId, RPC URL, and block explorer URL.
- The chain will appear automatically in the Omni-Deploy UI's chain selector and in ArtExplorer.

9.2 Factory ABI Reference

Function	Visibility	Parameters	Returns
deployDeterministic	public	name, symbol, uri, salt (bytes32)	address (new collection)
getDeployedNFTs	view	—	address[] (all collections)

9.3 Environment Variables

Variable	Required	Description
NEXT_PUBLIC_NFT_STORAGE_KEY	Yes	Pinata JWT for IPFS uploads

10

ROADMAP

Phase	Milestone	Status
v1.0 — Foundation	Factory contract + Omni-Deploy UI + ArtExplorer	Complete
v1.1 — Security	Server-side IPFS upload proxy + rate limiting	In progress
v1.2 — Indexing	Subgraph integration for real-time collection analytics	Planned Q2 2025
v2.0 — Governance	Factory upgrade governance via on-chain multisig	Planned Q3 2025
v2.1 — Royalties	ERC-2981 royalty standard support in collection contracts	Planned Q3 2025
v2.2 — Batch Mint	Gas-optimised batch minting with merkle-proof allowlists	Planned Q4 2025
v3.0 — LayerZero	Cross-chain NFT messaging for unified ownership across chains	Research

11 CONCLUSION

ArtMintfy demonstrates that multi-chain NFT deployment can be as simple as filling out a form. By abstracting the complexity of network switching, address derivation, and media pinning into a coherent four-step wizard, the protocol makes omni-chain presence accessible to every creator — not just those with smart contract engineering backgrounds.

The deterministic addressing model is ArtMintfy's most important technical contribution. A collection that exists at the same address on six chains is fundamentally different from six separate collections: it can be indexed, traded, and transferred with a single identifier, paving the way for a unified multi-chain NFT identity layer.

The protocol is intentionally minimal. The factory contract is under 100 lines of Solidity. The UI is a single Next.js application with no proprietary backend. This simplicity is a feature, not a limitation: it makes the protocol easy to audit, fork, and extend. Contributions are welcome at github.com/jerydam/artmintfy.

Source Code Open MIT-equivalent — No Rights Reserved	Audit Status Pending Professional audit scheduled for v1.1	Contact GitHub github.com/jerydam/artmintfy
--	--	--

ArtMintfy Whitepaper v1.0 — Built by Jerydam — © 2024 ArtMintfy. No Rights Reserved.